

Next Permutation

Company: JPMorgan Chase & Co.

Round: Online Assessment (OA)

Year: 2023

Difficulty: Medium

Topic: Arrays, Two-Pointer, In-place Manipulation

Source: [LeetCode 31 - Next Permutation](#)

Problem Description

Given an array of integers `nums`, rearrange its elements to obtain the next lexicographically greater permutation of the array.

- The new arrangement must be the smallest possible permutation that is greater than the current one.
- The replacement must be done in place and must use only constant extra memory.
- If the array is already in its maximum possible order, rearrange it into the minimum possible order, i.e., ascending order.

Input Format

- An integer array `nums`.
- The array contains `N` integers ($1 \leq N \leq 100$).

Output Format

Modify the given array in place such that:

- `nums` becomes the next greater permutation of its current arrangement.
- If no greater permutation is possible, rearrange `nums` in ascending order.

Sample Input / Output

Example 1

Input: `nums = [1, 2, 3]`

Output: `nums = [1, 3, 2]`

Example 2

Input: `nums = [3, 2, 1]`

Output: `nums = [1, 2, 3]`

Example 3

Input: `nums = [1, 1, 5]`

Output: `nums = [1, 5, 1]`

Explanation

In Example 1, among all six permutations of `[1, 2, 3]`, the permutation that comes immediately after `[1, 2, 3]` in lexicographic order is `[1, 3, 2]`.

In Example 2, `[3, 2, 1]` is the last (largest) permutation possible, so there is no greater one — the array wraps around to the smallest permutation, `[1, 2, 3]`.

In Example 3, swapping the last two elements produces the next greater arrangement, `[1, 5, 1]`.

Approach to Solve This Problem:

The key idea is to make the **smallest possible change from the right side** of the array. We first find the position where the permutation can be increased, then rearrange the remaining elements into the smallest possible order.

1. Find the pivot point:

- Start from the right side of the array and find the first index i such that $\text{nums}[i] < \text{nums}[i + 1]$.
- This is the position where we can make the permutation slightly larger.
- If no such index exists, the array is already the largest permutation, so simply reverse the entire array to get the smallest permutation.

2. Find the next greater element:

- Start from the end of the array and find the first index j such that $\text{nums}[j] > \text{nums}[i]$.
- Since we are scanning from the right, this gives the **smallest element greater than the pivot**

3. Swap the pivot and the next greater element:

- Swap $\text{nums}[i]$ and $\text{nums}[j]$.
- This makes the array slightly larger while keeping the change as small as possible.

4. Reverse the remaining part:

- Reverse the elements from $i + 1$ to the end of the array.
- The suffix is changed into ascending order, giving the **smallest possible arrangement** after the pivot.

Java Solution:

```
class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int i = n - 2;

        // 1. Find the first index from the right where nums[i] < nums[i+1]
        while (i >= 0 && nums[i] >= nums[i + 1]) {
            i--;
        }

        // 2. If such an index exists, find the rightmost successor
        //    greater than nums[i] and swap them
        if (i >= 0) {
            int j = n - 1;
            while (nums[j] <= nums[i]) {
                j--;
            }
            swap(nums, i, j);
        }

        // 3. Reverse the suffix starting at i + 1
        reverse(nums, i + 1, n - 1);
    }

    private void swap(int[] nums, int a, int b) {
```

```
        int temp = nums[a];
        nums[a] = nums[b];
        nums[b] = temp;
    }

    private void reverse(int[] nums, int start, int end) {
        while (start < end) {
            swap(nums, start, end);
            start++;
            end--;
        }
    }

    public static void main(String[] args) {
        Solution sol = new Solution();
        int[] nums = {1, 2, 3};
        sol.nextPermutation(nums);
        for (int num : nums) {
            System.out.print(num + " ");
        }
        // Output: 1 3 2
    }
}
```

Solution:

1. Locate the pivot: Traverse nums from right to left with index $i = n - 2$ downward, decrementing while $\text{nums}[i] \geq \text{nums}[i + 1]$. This finds the first index that breaks the non-increasing pattern from the right.
2. Handle the “no pivot” case implicitly: If i becomes -1 , the whole array is non-increasing; the code skips the swap step and goes straight to reversing the full array, which sorts it into ascending order.
3. Find and swap with the successor: If a pivot was found ($i \geq 0$), scan from the end ($j = n - 1$) leftward until $\text{nums}[j] > \text{nums}[i]$, then swap $\text{nums}[i]$ and $\text{nums}[j]$.
4. Reverse the suffix: Reverse nums from index $i + 1$ to $n - 1$ using a standard two-pointer swap, converting the non-increasing suffix into ascending order and completing the next permutation.

Complexity Analysis:

Time Complexity: $O(n)$

Each of the three phases — finding the pivot, finding the successor, and reversing the suffix — makes at most one linear pass over the array, so the overall time complexity is $O(n)$.

Space Complexity: $O(1)$

The array is rearranged using only a constant number of index variables (i , j , and the pointers used for swapping/reversing); no auxiliary array or data structure is used, satisfying the in-place, constant-extra-memory requirement.